Descriptive Statistics

Session Objectives:

- Understand the concepts of range and interquartile range (IQR) and calculate them from a given dataset.
- Compute the variance and standard deviation and interpret their significance in data dispersion.
- Recognise and interpret skewness and kurtosis to understand the shape and distribution of data.
- Apply descriptive statistical measures to real-world data and derive meaningful insights.
- Compare and contrast different measures of dispersion and central tendency.
- Utilise statistical software/Python to perform these calculations efficiently.
- Interpret and communicate the results of descriptive statistical analysis clearly.

```
# Dataset Characteristics
Total Customers: 1,000
Store Locations: North, South, East, West
Satisfaction Ratings: 1-5 scale
Income Range: $20,000 - $80,000
```

```python
# Sample Dataset Vs Population Dataset [Statistical Analysis]
# Population
import numpy as np
import pandas as pd

np.random.seed(101) # reproducibility
df_population = pd.DataFrame({
    'CustomerID' : range(1,1001), # [1,...1000]
    'Store_Location' : np.random.choice(['North', 'South', 'East', 'West'] , 1000),
    'SatisfactionRatings' : np.random.randint(1,6,1000),
    'IncomeRange' : np.random.randint(20000,80000,1000)
})
df_population
```

	CustomerID	Store_Location	SatisfactionRatings	IncomeRange
0	1	West	4	29821
1	2	West	4	54863
2	3	South	4	50863
3	4	East	2	20584
4	5	West	1	25188
...	...	...	...	...
995	996	East	4	20202
996	997	East	1	33147
997	998	South	4	21979
998	999	South	1	34678
999	1000	East	4	44089

1000 rows × 4 columns

```python
# Random Sampling
random_sample = df_population.sample(n = 100, random_state = 121)
random_sample
```

	CustomerID	Store_Location	SatisfactionRatings	IncomeRange
380	381	West	3	77721
288	289	North	5	47252
328	329	West	5	61553
208	209	South	2	50830
284	285	South	5	22997
...	...	...	...	...
405	406	East	2	62035
993	994	West	5	46594
230	231	East	5	45013
989	990	South	1	22080
421	422	North	4	23585

100 rows × 4 columns

```python
# Stratified_Sampling
stratified_sample = (
    df_population
    .sample(frac = 1 , random_state = 77)
    .groupby('Store_Location')
    .tail(25)
)
stratified_sample
```

	CustomerID	Store_Location	SatisfactionRatings	IncomeRange
678	679	North	4	57356
944	945	North	1	35064
838	839	North	4	28584
824	825	North	1	57718
507	508	North	4	72169
...	...	...	...	...
235	236	North	5	54452
980	981	West	1	75513
596	597	South	5	46675
607	608	East	4	56971
727	728	East	1	73498

100 rows × 4 columns

```python
stratified_sample['Store_Location'].value_counts()
```

```
Store_Location
North    25
East     25
South    25
West     25
Name: count, dtype: int64
```

```python
# Cluster Sampling
clusters = df_population['Store_Location'].unique()
clusters
```

```
array(['West', 'South', 'East', 'North'], dtype=object)
```

```python
selected_clusters = np.random.choice(clusters , size = 2, replace = False)
selected_clusters
```

```
array(['West', 'North'], dtype=object)
```

```python
cluster_sampling = df_population[df_population['Store_Location'].isin(selected_clusters)]
cluster_sampling
```

	CustomerID	Store_Location	SatisfactionRatings	IncomeRange
0	1	West	4	29821
1	2	West	4	54863
4	5	West	1	25188
5	6	West	1	30172
7	8	West	4	74736
...	...	...	...	...
987	988	North	2	77799
990	991	West	5	25148
991	992	West	1	32795
993	994	West	5	46594
994	995	North	3	36427

506 rows × 4 columns

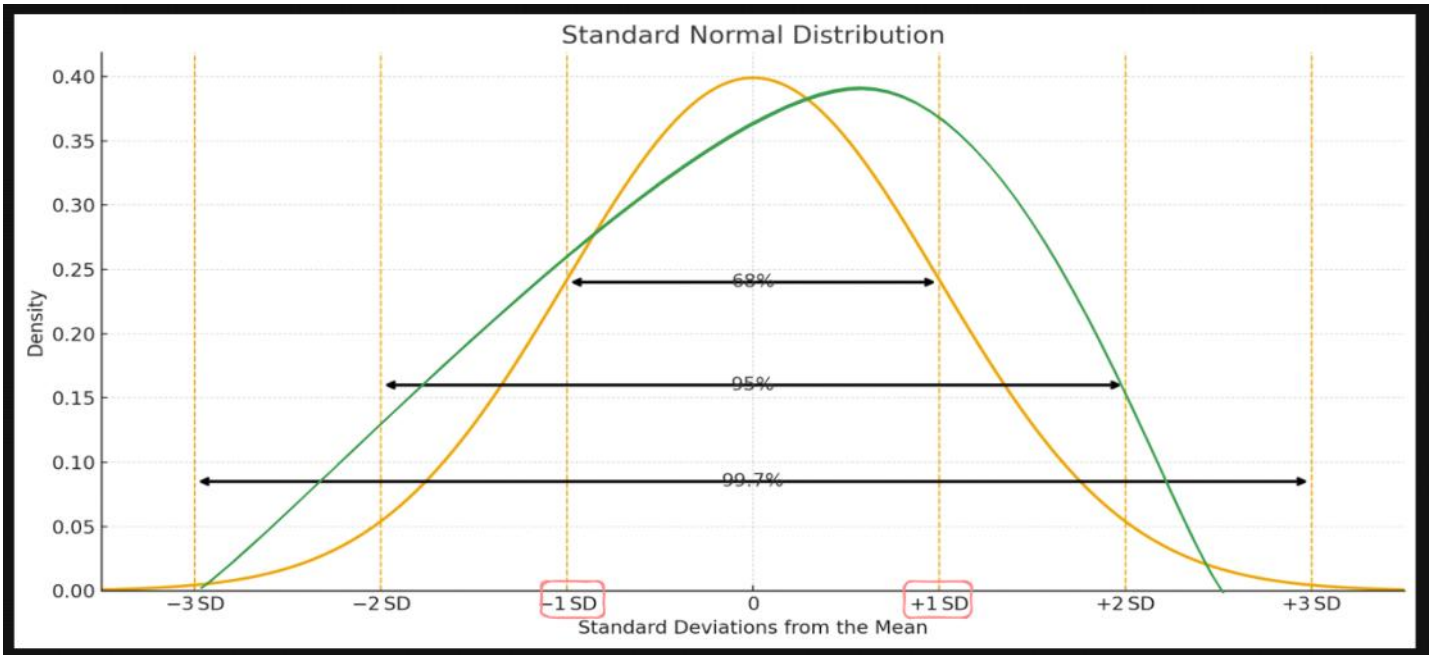
```python
# Systematic Sampling -> Kth factor -> N/n -> 1000/100 -> 10th
systematic_sampling = df_population.iloc[::10 , :] # Positional Based Indexing
systematic_sampling
```

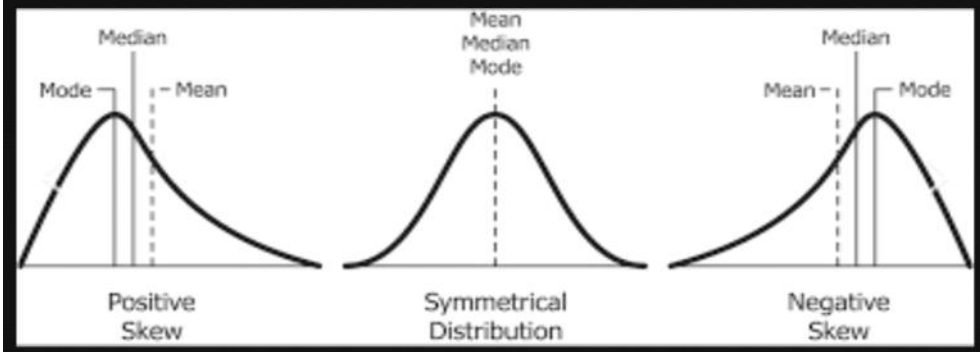
	CustomerID	Store_Location	SatisfactionRatings	IncomeRange
0	1	West	4	29821
10	11	South	2	32499
20	21	South	3	46522
30	31	South	1	33871
40	41	North	3	40586
...	...	...	...	...
950	951	East	5	54419
960	961	South	1	56185
970	971	South	2	47377
980	981	West	1	75513
990	991	West	5	25148

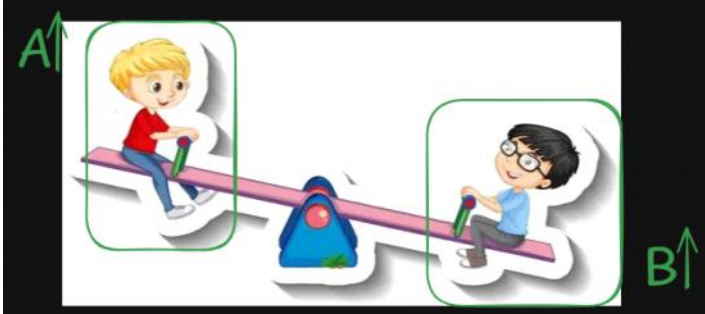
100 rows × 4 columns

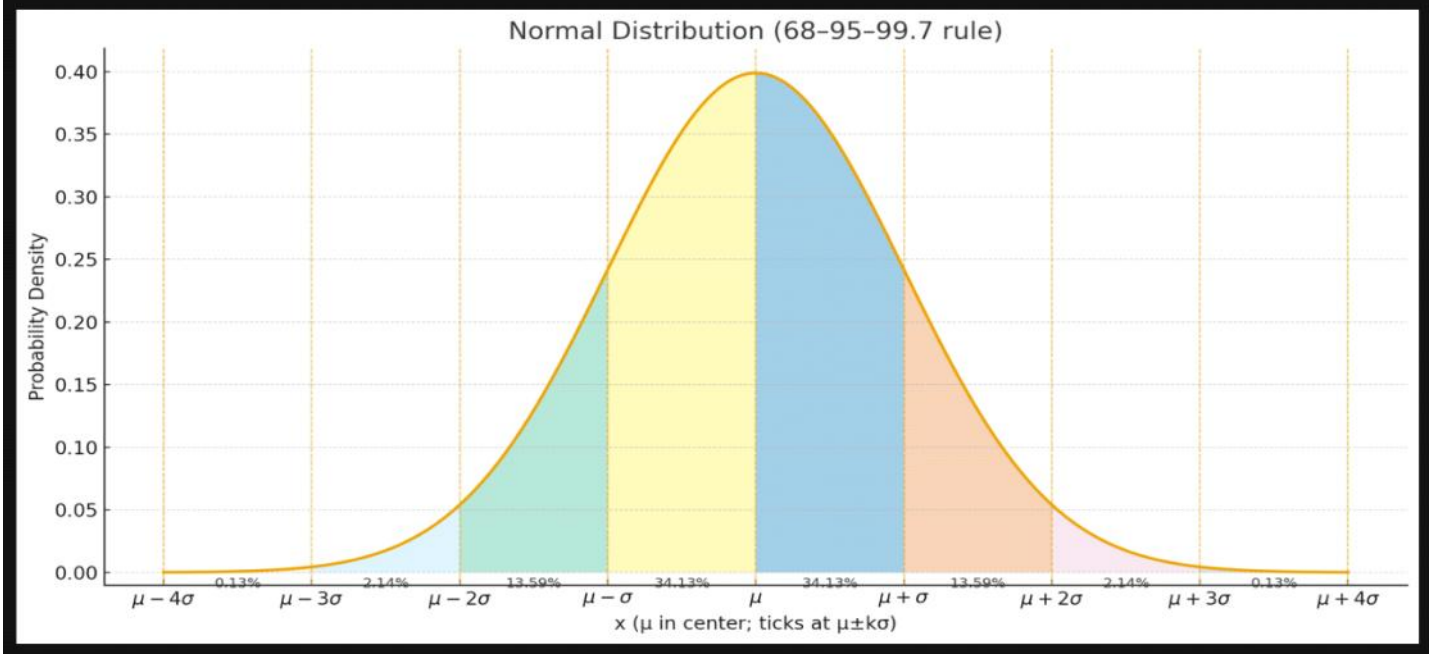
```python
# Evaluating the Population Mean Vs Sample Mean from various Sampling Techniques

actual_mean_income = df_population['IncomeRange'].mean()
print(f"Actual Mean: {actual_mean_income}")

# Random Sample Technique
random_sample_mean_income = random_sample['IncomeRange'].mean()
print(f"Random Sample Mean: {random_sample_mean_income}")

# Stratified Sample Technique
stratified_sample_mean_income = stratified_sample['IncomeRange'].mean()
print(f"Stratified Sample Mean: {stratified_sample_mean_income}")

# Systematic Sample Technique
systematic_sample_mean_income = systematic_sampling['IncomeRange'].mean()
print(f"Systematic Sample Mean: {systematic_sample_mean_income}")

# Cluster Sample Technique
cluster_sample_mean_income = cluster_sampling['IncomeRange'].mean()
print(f"Cluster Sample Mean: {cluster_sample_mean_income}")
```

```
Actual Mean: 50484.642
Random Sample Mean: 51029.3
Stratified Sample Mean: 51351.1
Systematic Sample Mean: 49200.4
Cluster Sample Mean: 50900.16205533597
```


Standard Normal Distribution
Density
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
68%
95%
99.7%
−3 SD
−2 SD
−1 SD
0
+1 SD
+2 SD
+3 SD
Standard Deviations from the Mean


Median
Mode
Mean
Positive Skew
Mean
Median
Mode
Symmetrical Distribution
Median
Mean
Mode
Negative Skew


A
B


Normal Distribution (68–95–99.7 rule)
Probability Density
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.13%
2.14%
13.59%
34.13%
34.13%
13.59%
2.14%
0.13%
μ − 4σ
μ − 3σ
μ − 2σ
μ − σ
μ
μ + σ
μ + 2σ
μ + 3σ
μ + 4σ
x (μ in center; ticks at μ±kσ)

$$Skewness = \frac{n}{(n-1)(n-2)} \sum_{i=1}^{n} \left(\frac{x_i - \bar{x}}{s} \right)^3$$

Negative Skewness

Left-skewed: tail extends to the left, most values cluster on the right

Zero Skewness

Symmetric: perfectly balanced distribution around the center

Positive Skewness

Right-skewed: tail extends to the right, most values cluster on the left

```python
# Descriptive Statistics -> Skewness
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import skew
```

```python
dir(skew)
```

```
['__annotations__',
 '__builtins__',
 '__call__',
 '__class__',
 '__closure__',
 '__code__',
 '__defaults__',
 '__delattr__',
 '__dict__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__get__',
 '__getattribute__',
 '__getstate__',
```

```python
print(skew.__doc__)
```

```
Compute the sample skewness of a data set.

For normally distributed data, the skewness should be about zero. For
unimodal continuous distributions, a skewness value greater than zero means
that there is more weight in the right tail of the distribution. The
function `skewtest` can be used to determine if the skewness value
is close enough to zero, statistically speaking.

Parameters
----------
a : ndarray
    Input array.
axis : int or None, default: 0
    If an int, the axis of the input along which to compute the statistic.
    The statistic of each axis-slice (e.g. row) of the input will appear in a
    corresponding element of the output.
    If ``None``, the input will be raveled before computing the statistic.
```

```
dir(np.random)
```

```
 'beta',
 'binomial',
 'bit_generator',
 'bytes',
 'chisquare',
 'choice',
 'default_rng',
 'dirichlet',
 'exponential',
 'f',
 'gamma',
 'geometric',
 'get_bit_generator',
 'get_state',
 'gumbel',
 'hypergeometric',
 'laplace',
```

```
# Positive Skewness
np.random.seed(42) # Reproducibility
income_distribution = np.random.gamma(shape = 2.0, scale = 1000, size = 1000) # Right Skewed
income_distribution
```

```
array([2393.67938987, 1494.46473022, 1382.28358437, 1382.30229433,
       4649.71441223, 2866.70623096, 1131.07800174, 2469.81447343,
       1998.96026406,  215.91494349,  671.24228269, 2106.16563735,
       4365.17686748, 1391.85218496, 1058.02743905, 1814.00754147,
       1005.28448072, 1317.74138114, 1649.30247894,  640.13730484,
       2970.06009421,  535.18144628,  472.79344383, 1933.95041789,
       2813.35380798, 1897.83557436, 1521.77917708, 1307.38143498,
       1139.90983069, 3437.79930052, 2121.04393939, 1217.31881366,
        936.60652931, 2587.61895671, 3383.4416734 , 3181.2097818 ,
        801.04704821, 1298.49780129, 1121.43496752, 1438.28811579,
       2951.08852336, 4102.26245721, 2178.48714031,  964.84462309,
       1620.84196489, 4612.53953507, 1781.58860155, 1309.68470258,
       1398.8526118 , 2171.51336405,  825.63173451, 1099.19711812,
       3149.77239357, 2128.12761532, 1795.16084048, 3256.01356468,
        914.68578378, 1278.43491723, 2078.93030478, 2026.9145906 ,
        425.8455743 , 1180.46141361, 1466.99837036, 2244.60554203,
       2021.76364152, 1572.392964  ,  214.10508191, 1632.67119445,
```

```
skew_value = skew(income_distribution)
skew_value
```

```
1.1962783386596858
```

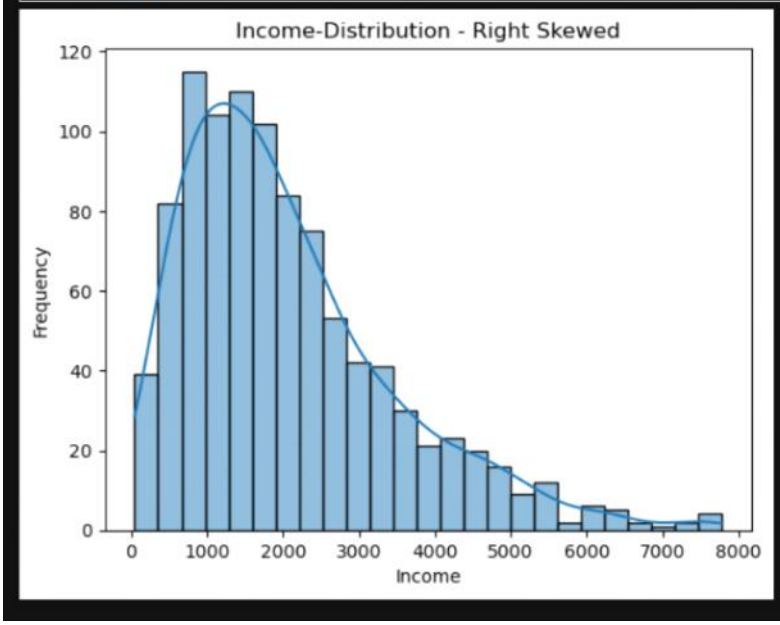
```
# Positive Skewness
np.random.seed(42) # Reproducibility
income_distribution = np.random.gamma(shape = 2.0, scale = 1000, size = 1000) # Right Skewed
income_distribution
```

```
array([2393.67938987, 1494.464
       4649.71441223, 2866.70
       1998.96026406,  215.91
       4365.17686748, 1391.85
       1005.28448072, 1317.74
       2970.06009421,  535.18
       2813.35380798, 1897.83
       1139.90983069, 3437.79
        936.60652931, 2587.61
        801.04704821, 1298.49
       2951.08852336, 4102.26
       1620.84196489, 4612.53
       1398.8526118 , 2171.51
```

```
Docstring:
gamma(shape, scale=1.0, size=None)

Draw samples from a Gamma distribution.

Samples are drawn from a Gamma distribution with specified parameters,
`shape` (sometimes designated "k") and `scale` (sometimes designated
"theta"), where both parameters are > 0.

.. note::
    New code should use the `~numpy.random.Generator.gamma`
    method of a `~numpy.random.Generator` instance instead;
    please see the :ref:`random-quick-start`.

Parameters
```

```
Parameters
----------
shape : float or array_like of floats
    The shape of the gamma distribution. Must be non-negative.
scale : float or array_like of floats, optional
    The scale of the gamma distribution. Must be non-negative.
    Default is equal to 1.
size : int or tuple of ints, optional
    Output shape.  If the given shape is, e.g., ``(m, n, k)``, then
    ``m * n * k`` samples are drawn.  If size is ``None`` (default),
    a single value is returned if ``shape`` and ``scale`` are both
scalars.
    Otherwise, ``np.broadcast(shape, scale).size`` samples are drawn.


Notes
-----
The probability density for the Gamma distribution is

.. math:: p(x) = x^{k-1}\frac{e^{-x/\theta}}{\theta^k\Gamma(k)},

where :math:`k` is the shape and :math:`\theta` the scale,
and :math:`\Gamma` is the Gamma function.

The Gamma distribution is often used to model the times to failure of
electronic components, and arises naturally in processes for which the
waiting times between Poisson distributed events are relevant.
```

```python
# Visualization [Histogram , with KDE]
sns.histplot(income_distribution , bins = 25 , kde = True)
plt.title("Income-Distribution - Right Skewed")
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.show()
```



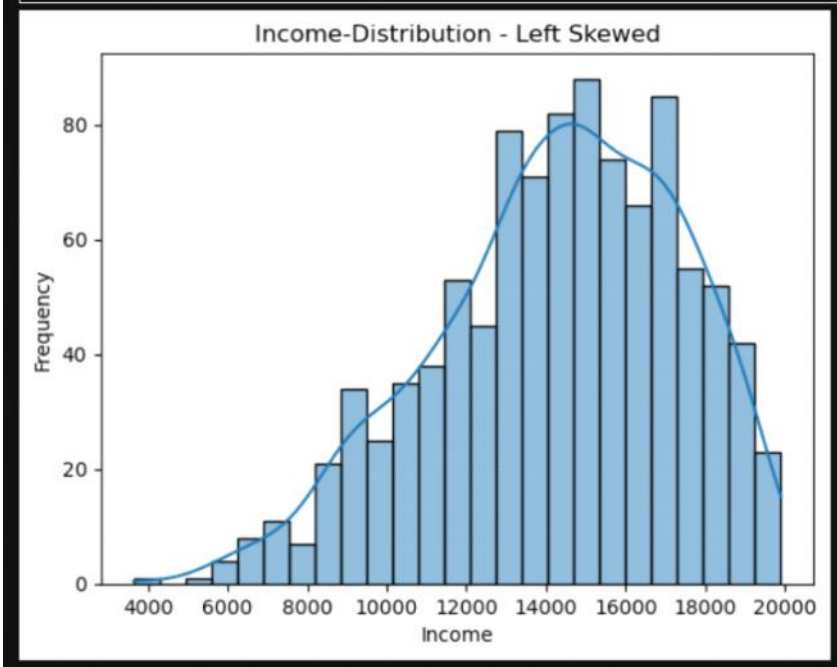
```
# Left Skewed [Negative Skewed] -> .beta
max_income = 20000
income_distribution = np.random.beta(a = 5 , b = 2 , size = 1000) * max_income
income_distribution
```

```
array([11784.8151352 , 18460.22476963, 16442.28354515,  9585.79618372,
       17099.88947283, 16273.21648418, 17949.28975731, 13715.98895158,
       13299.87962456, 14896.67994954, 12142.74918074, 14257.0938715 ,
       13724.49807398, 14498.665256  , 15464.84544882, 11801.0440922 ,
       19019.80261647, 17673.66530194, 15131.3167427 ,  8904.77337416,
       17196.24488801, 11957.14407556, 19262.16184348, 13855.03194714,
       11651.22030157, 13179.74687304, 13915.00216194, 14193.50354784,
       11574.67250981, 15606.06591889, 15681.8390013 ,  9354.43817102,
       16658.36903872, 13146.31581928, 11537.87336196, 14408.1695026 ,
       15244.59755295,  7117.18761407, 18711.43187004, 11354.95085787,
       14174.95216338, 13811.55449724, 16317.84667653, 16655.08131364,
       15448.46807177,  6306.28590165,  9148.27154154, 15764.92640306,
       13796.83157057, 14839.9450803 , 15740.36932859, 16386.91282242,
       17018.63095834, 13558.99797202, 13554.5931736 , 13345.75790742,
       15002.7303979 , 16688.05886141, 15882.770333  , 18954.26002372,
       13163.62127235, 19090.24664057, 10358.84462025, 12368.05241406,
        7502.37559703, 11064.52464867, 16447.11092223, 14113.17123984,
```

```
skew_value = skew(income_distribution)
skew_value
```

```
-0.4731157042145485
```

```
# Visualization [Histogram , with KDE]
sns.histplot(income_distribution , bins = 25 , kde = True)
plt.title("Income-Distribution - Left Skewed")
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.show()
```



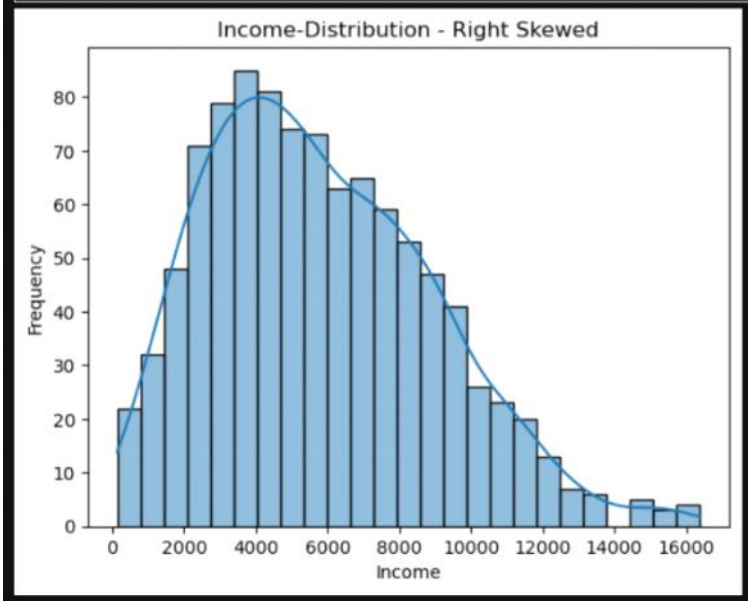
```python
# Right Skewed [Positive Skewed] -> .beta
max_income = 20000
income_distribution = np.random.beta(a = 2 , b = 5 , size = 1000) * max_income
income_distribution
```

```
array([10023.73223821,  4929.36838697,  8553.1458431 ,  1490.17308327,
        3598.10141617,  6791.72063099,  1026.93172643,  3325.39648579,
        3087.43669088,  6840.06242023,  6284.77674117,  9770.01909058,
         875.12274173,  2999.99449696,  4503.75960278, 12578.2596267 ,
        5818.25107545,  6388.1503831 ,  4069.21682369, 11292.04272459,
        8421.69453265,  5135.74338786,  7464.73471326,  7455.20415464,
        5425.81145505,  9659.53477934,  2141.41285658,  8352.24951376,
        3883.81458556,  5144.20817067,  8939.3303469 ,  4750.23911976,
        3765.90585965,  3167.18633077,  6219.12732076,  2798.41281289,
        6943.30972999,  7872.51862367,  2391.61731347,  3805.01244941,
        8418.80004901,  3984.1392443 ,  2637.67034959,  4253.49293326,
        8397.02638936,  4882.73355032,  3126.27452114,  8649.73989652,
       10580.29094291,  5914.15861857,  2714.77574068,  7099.46098342,
        4815.8777145 ,  3307.96185021,  2296.77354078,  4347.60154397,
        8220.55254807,  5346.92925589,  8385.76244422, 10580.03077823,
        2603.1369897 ,  2297.27938653,  2347.66326858,  3923.40615823,
        8982.77291647, 12203.39639081,  7632.6847462 ,  3319.92956302,
```

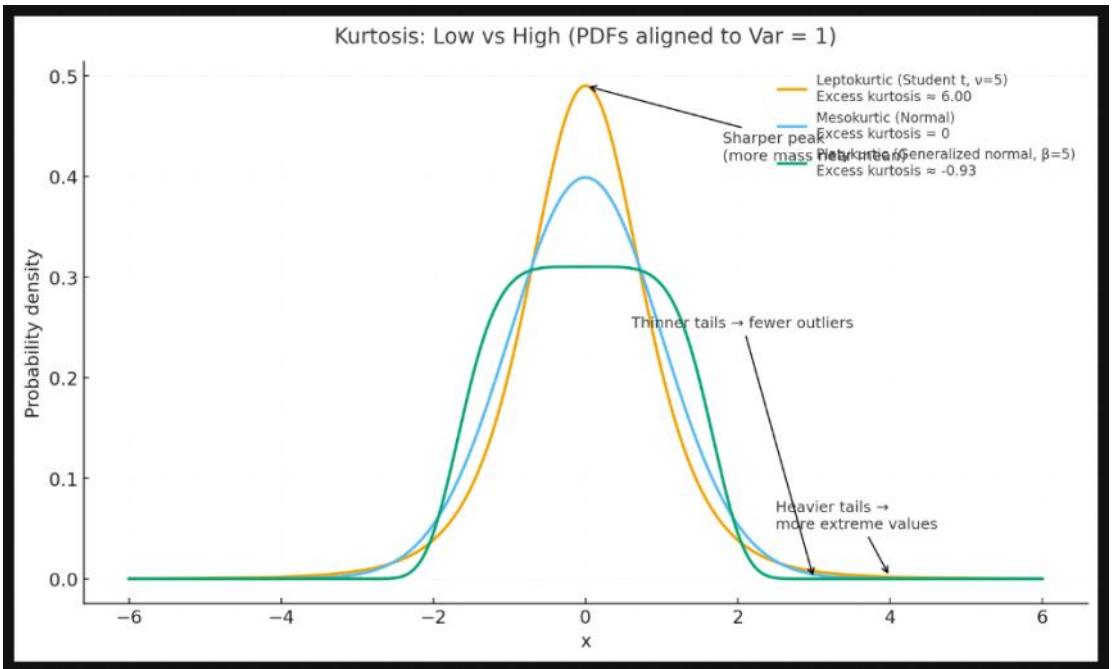
```python
skew_value = skew(income_distribution)
skew_value
```

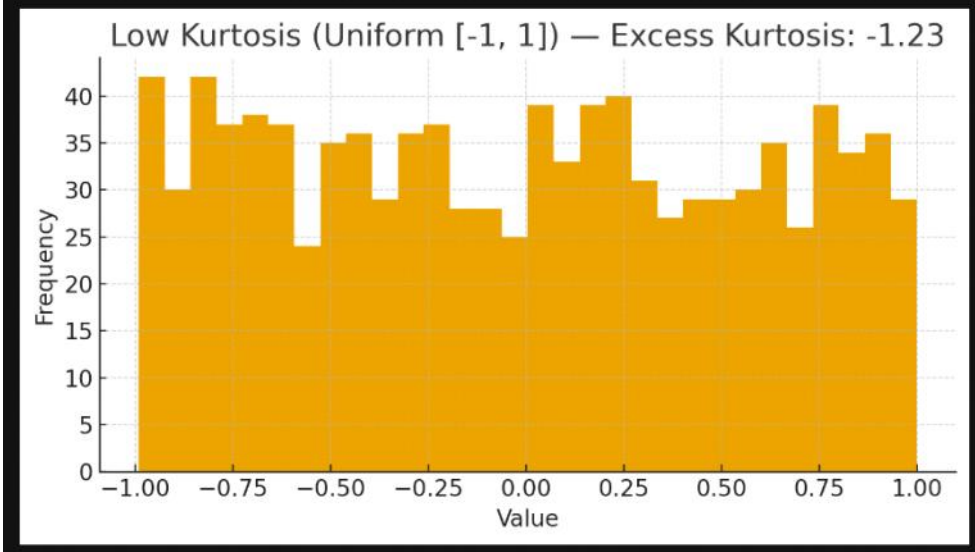
```
0.56625214164857
```

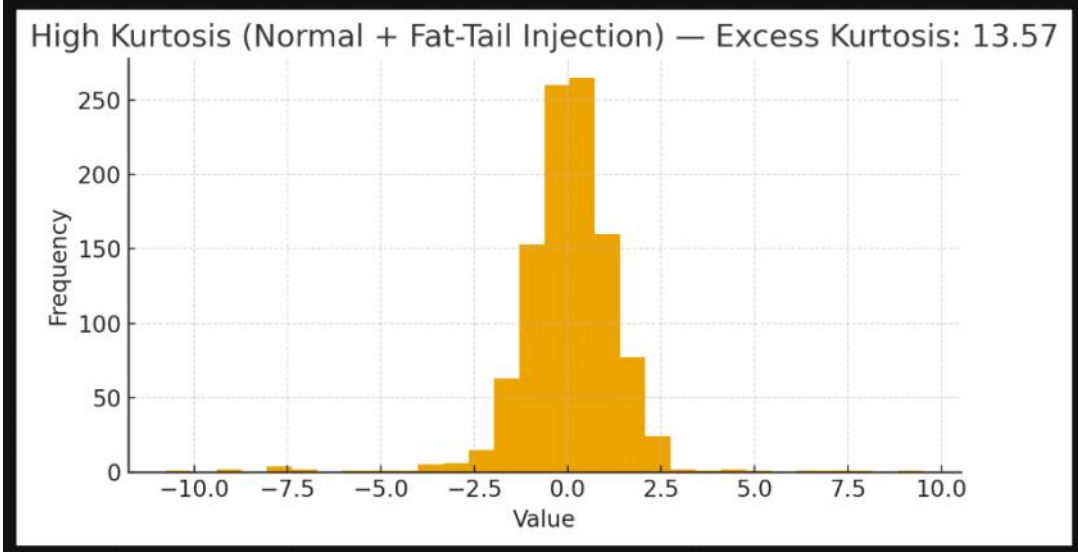
```python
# Visualization [Histogram , with KDE]
sns.histplot(income_distribution , bins = 25 , kde = True)
plt.title("Income-Distribution - Right Skewed")
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.show()
```

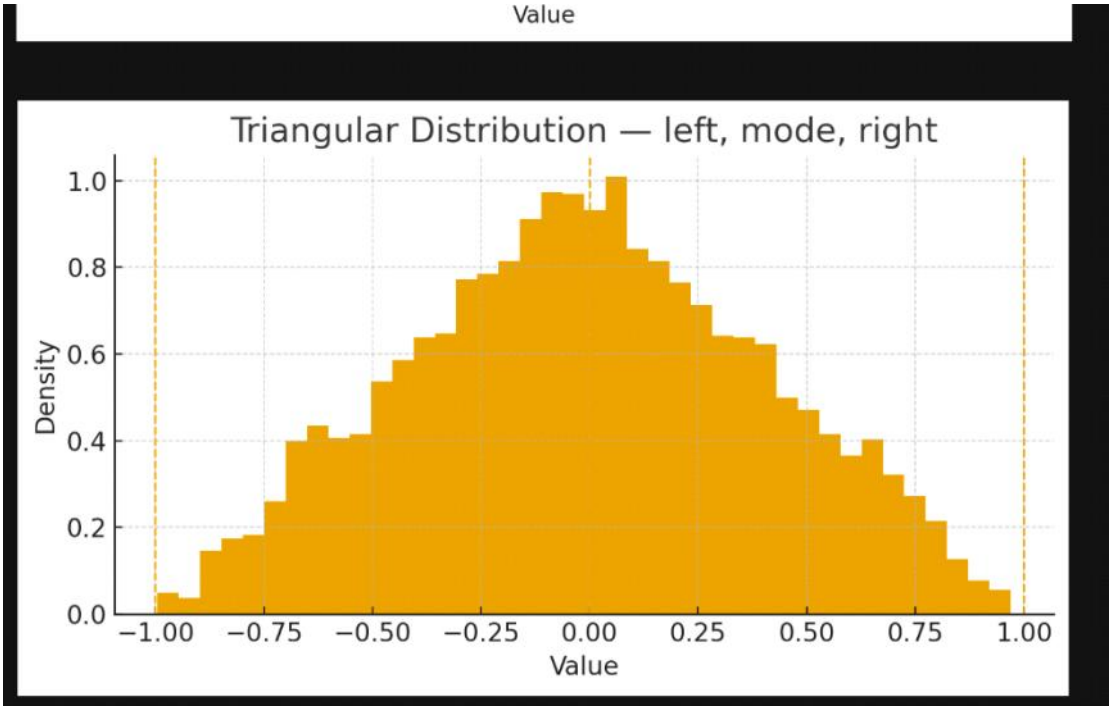


Kurtosis is a statistical measure that describes the **degree of "tailedness"** in the probability distribution of a real-valued random variable. It provides insight into the shape of the distribution, particularly in terms of the presence of outliers. There are different types of kurtosis, such as **leptokurtic** (high kurtosis, indicating heavy tails) and **platykurtic** (low kurtosis, indicating light tails). Additionally, **kurtosis risk** refers to the risk that arises when a statistical model assumes a normal distribution but is applied to data that may have extreme values. Wikipedia +1


Kurtosis: Low vs High (PDFs aligned to Var = 1)
Leptokurtic (Student t, ν=5)
Excess kurtosis ≈ 6.00
Mesokurtic (Normal)
Excess kurtosis = 0
Excess kurtosis ≈ -0.93
Sharper peak
Thinner tails → fewer outliers
Heavier tails →
more extreme values
Probability density
x


Low Kurtosis (Uniform [-1, 1]) — Excess Kurtosis: -1.23
Frequency
Value


High Kurtosis (Normal + Fat-Tail Injection) — Excess Kurtosis: 13.57
Frequency
Value


Value
Triangular Distribution — left, mode, right
Density
1.0
0.8
0.6
0.4
0.2
0.0
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
Value